STUDENT PROJECT BRIEF

Robust NLP Under Character Noise

A reproducible study of tokenization and representation stability, reported as an ACL-style paper.

TEAM	HORIZON	METHOD	FINAL OUTPUT
2 students	Semester	Paired clean/noisy study	ACL-style paper + repo

CORE RULE Corrupt characters in clean text before WordPiece tokenization. Keep a matched clean branch and hold the tokenizer, model checkpoint, examples, and evaluation procedure constant.

1. Purpose

This project asks how controlled character-level noise changes a BERT-compatible NLP system: first the subword sequence, then the encoder representation, and finally task or language-model performance. The goal is an evidence-backed robustness claim—not merely a working demo.

2. Driving research question

QUESTION How do noise type and intensity affect WordPiece tokenization, masked-encoder representations, and downstream performance, and are those effects predictable across data or model scale?

Learning outcomes

- Implement a deterministic, configurable character-corruption pipeline.
- Use unit-cost Levenshtein distance correctly and validate it with known examples.
- Explain and operationally distinguish BPE from WordPiece.
- Design paired experiments with leakage-resistant train/validation/test controls.
- Report uncertainty, limitations, ethical considerations, and reproducibility evidence.
- Write a concise NLP paper using the official ACL publication template.

RESEARCH POSTURE Treat power-law behavior as a testable hypothesis inspired by prior scaling-law work—not as an expected or guaranteed result.

RESEARCH PROJECT

Experimental architecture

One source example produces a clean control and one or more noisy variants.

3. Correct paired pipeline

Clean branch



Noisy branch



ORDER CHECK If the intervention occurs after WordPiece, it is token-level corruption. That may be a valid follow-up experiment, but it must not be labeled character noise.

4. Noise model

Operation	Example	Required record
Substitution	noise → no1se	source character, replacement rule
Deletion	noise → nose	deleted index and character
Insertion	noise → noiise	inserted character and index
Optional transposition	noise → nosie	adjacent indices; report separately from Levenshtein operations

Experimental controls

- Fix random seeds and persist a corruption manifest for every example.
- Stratify by noise type and intensity; do not silently mix regimes.
- Protect required special tokens and document whitespace/Unicode normalization.
- Use validation data to select settings; reserve the test set for final estimates.

RESEARCH PROJECT

Minimum Edit Distance — corrected standard

Use equal weights of 1 for insertion, deletion, and substitution.

5. Unit-cost Levenshtein distance

Operation	Cost	Interpretation
Insertion	1	Add one character
Deletion	1	Remove one character
Substitution	1	Replace one character with another

RECURRENCE $D[i,j] = \min(D[i-1,j] + 1, D[i,j-1] + 1, D[i-1,j-1] + [s_i \neq t_j])$; initialize $D[i,0] = i$ and $D[0,j] = j$.

Reference matrix: intention → execution

	#	e	x	e	c	u	t	i	o	n
#	0	1	2	3	4	5	6	7	8	9
i	1	1	2	3	4	5	6	6	7	8
n	2	2	2	3	4	5	6	7	7	7
t	3	3	3	3	4	5	5	6	7	8
e	4	3	4	3	4	5	6	6	7	8
n	5	4	4	4	4	5	6	7	7	7
t	6	5	5	5	5	5	5	6	7	8
i	7	6	6	6	6	6	6	5	6	7
o	8	7	7	7	7	7	7	6	5	6
n	9	8	8	8	8	8	8	7	6	5

VALIDATION The bottom-right cell is 5, so the unit-cost Levenshtein distance between intention and execution is 5. Boundary values are non-negative prefix lengths.

RESEARCH PROJECT

BPE and WordPiece

Related subword strategies, different vocabulary-learning rules.

6. Make the distinction explicit

Question	Byte-Pair Encoding (BPE)	WordPiece
Shared idea	Begin with small units and build reusable subwords.	Begin with small units and build reusable subwords.
Selection rule	Merge the most frequent adjacent pair.	Choose vocabulary additions using a likelihood/association-style score rather than raw pair frequency alone.
Visible convention	Implementation-specific boundary markers.	BERT tokenizers commonly show continuation pieces with ##.
Role in this course	Conceptual subword-induction example in the lecture.	Required tokenizer for the BERT robustness experiment.

IMPLEMENTATION NOTE Use a documented BERT-compatible WordPiece implementation or a carefully specified implementation of your own. Devlin et al. identify WordPiece but do not fully define every vocabulary-training detail; record library, version, normalization, casing, and vocabulary source.

7. WordPiece requirements

- Reserve [PAD], [UNK], [CLS], [SEP], and [MASK] as required by the selected BERT checkpoint.
- Do not corrupt special tokens or insert noise after their token IDs are created.
- Choose either a fixed pretrained tokenizer or a tokenizer trained on course data; justify the choice and do not change it between clean/noisy branches.
- Save vocabulary, tokenizer configuration, normalization rules, and checksum with the repository.

Tokenization diagnostics

Diagnostic	What it reveals
Token count / fertility	Whether noise fragments words into more pieces
Token-sequence change rate	How often noise changes the subword representation
[UNK] rate	Whether the tokenizer loses coverage
Continuation-piece rate	Whether internal fragments increase
Example-level alignment	Which character edits trigger the largest token shifts

RESEARCH PROJECT

Experimental plan and evaluation

Move from data integrity to paired estimates, then to defensible claims.

8. Data protocol

Split	Role	Restriction
Training	Fit any trainable component or fine-tuning objective	Never use test outcomes to revise the pipeline
Validation	Choose noise rates, model settings, and reporting thresholds	Track every decision
Test	Final clean/noisy comparison	Evaluate only after the analysis plan is frozen

WikiText-2 is appropriate for pipeline development. WikiText-103 may support a larger benchmark when compute and time permit. Dataset scale does not replace careful controls, and model context length must match the selected checkpoint rather than an assumed 1,024-token window.

9. Measures

Layer	Primary measures	Reporting expectation
Corruption	MED, edit counts, noise rate	Distribution by operation and severity
Tokenizer	fertility, change rate, [UNK], continuation pieces	Paired clean/noisy deltas
Representation	cosine similarity or centered distance	Layer-wise summary with uncertainty
Masked language model	masked-token cross-entropy; optional documented pseudo-perplexity	Do not label a BERT score as standard left-to-right perplexity
Downstream task	accuracy/F1 or task-appropriate metric	Absolute score and robustness drop

Analysis standard

- Report paired effect sizes and confidence intervals, not only averages.
- Plot robustness curves across noise intensity and show per-noise-type results.
- Predefine exclusions and failed-run handling; keep failures visible in the audit log.
- Use a scaling-law fit only when the number and range of scale points can support it; compare with simpler baselines.

RESEARCH PROJECT

Milestones and reproducibility

Each phase ends with inspectable evidence, not just completed code.

10. Semester progression

Phase	Focus	Evidence checkpoint
1 — Foundations	corpus audit, corruption functions, MED tests, tokenizer decision	tests + data card + 20 paired examples
2 — Baseline	clean branch and stable evaluation harness	reproducible baseline table
3 — Robustness	noise grid and paired evaluation	curves + uncertainty + error slices
4 — Scale / extension	additional data/model scale or targeted ablation	model comparison or ablation memo
5 — Publication	analysis freeze, paper, repository, presentation	ACL-style manuscript + release checklist

11. Reproducibility record

Record	Minimum content
Environment	Python/library versions, hardware, package lockfile
Data	source, license, split checksums, preprocessing hash
Tokenizer	name/version or vocabulary checksum; normalization and casing
Corruption manifest	seed, operation, rate, locations, clean/noisy IDs
Model	checkpoint, configuration, fine-tuning settings
Runs	command/config, seed, status, metrics, artifacts

MINIMUM STANDARD A classmate should be able to recreate one headline table or figure from a fresh checkout using the documented command and fixed configuration.

RESEARCH PROJECT

ACL-style publication standard

Use the official Association for Computational Linguistics template and conventions.

12. Format authority

Official style files: https://github.com/acl-org/acl-style-files

The official template is authoritative for page geometry, columns, type sizes, title/author block, captions, references, anonymity when applicable, and final-file checks. Use the course-specified page limit; if none is announced, plan a concise 6–8 pages of main content plus references.

13. Required manuscript structure

Section	Purpose
Abstract (≤200 words)	Question, method, principal result, and significance
1 Introduction	Motivation, research question, contributions
2 Related Work	Robustness, noise, tokenization, masked encoders, scaling
3 Data and Noise Model	corpus, splits, corruption design, integrity controls
4 Methods	tokenizer, model, hypotheses, algorithms
5 Experimental Setup	conditions, metrics, statistics, compute
6 Results	tables/figures tied to hypotheses
7 Discussion	mechanisms, errors, alternative explanations
8 Conclusion	supported takeaways; no new claims
Limitations	required after the conclusion and before references
Ethical Considerations	include when applicable; discuss data, bias, misuse, and resources
References	complete, consistent, and linked to primary sources where possible

Publication-quality evidence

- Every table and figure must be readable without the surrounding prose and must state clean/noisy conditions.
- Report negative, null, or mixed findings with the same care as positive results.
- Separate observation from inference; do not describe proxy metrics as direct measures of human language competence.

RESEARCH PROJECT

Deliverables, checklist, and assessment

Submit a coherent research package whose claims can be traced to evidence.

14. Final package

Artifact	Required content
ACL-style PDF	complete manuscript, limitations, references, readable figures/tables
Repository	source, tests, environment, configurations, README, license notes
Data/manifest package	derived IDs or permitted artifacts, checksums, corruption records
Results package	machine-readable metrics and figure-generation inputs
Presentation	research question, method, one headline result, limits, next step

15. Submission checklist

☐ The noisy branch corrupts characters before WordPiece tokenization.

☐ Insertion, deletion, and substitution each cost 1 in MED calculations.

☐ The paper distinguishes BPE from WordPiece and documents the tokenizer implementation.

☐ Clean/noisy comparisons are paired and use identical model/evaluation settings.

☐ Test data were not used to tune the pipeline.

☐ Headline claims include uncertainty and are supported by saved results.

☐ ACL formatting, Limitations, citations, captions, and repository links are complete.

16. Assessment criteria

Criterion	Evidence of strong work
Research framing	specific, motivated, falsifiable question grounded in literature
Technical correctness	valid pipeline, MED, tokenizer distinction, and controls
Evaluation	paired design, appropriate measures, uncertainty, and error analysis
Interpretation	claims match results; alternatives and limitations are explicit
Reproducibility	auditable data, configurations, code, and generated artifacts
Publication	clear ACL-style narrative, polished figures/tables, complete references

Dual-Use, Ethics, and Responsible Testing

Character-level perturbations have legitimate uses in evaluating NLP robustness, accessibility, and performance on naturally noisy text. They may also have dual-use implications: similar perturbations can conceal malicious content, evade moderation or classification systems, or serve as an obfuscation layer for prompt-injection attempts.

The core project examines a BERT-compatible encoder and therefore does not directly test prompt injection against an instruction-following language model. Students may discuss prompt injection as a potential downstream security implication, but they must distinguish it from the character-noise and adversarial-evasion behavior measured in the primary experiment.

All testing must use authorized, controlled systems and datasets. Students must not probe third-party services, attempt to bypass production safeguards, or develop operational jailbreaks. Any security-oriented extension should use harmless proxy instructions in a closed environment and emphasize detection, normalization, robustness, and mitigation.

The final paper must:

- identify legitimate and potentially harmful uses of the technique;
- define the systems and threat model that are—and are not—being studied;
- describe safeguards used during experimentation;
- avoid publishing unnecessarily actionable bypass procedures;
- evaluate defensive measures and their false-positive consequences;
- discuss limitations, affected stakeholders, and responsible future work.